Assignment 2 – Developing a Highly Available Photo Album Website

Julian Hess
[redacted]
Lab Time: Tuesday 6:30pm

I. Introduction

This assignment is a continuation of Assignment 1B, where a photo album website was deployed. In assignment 2, the aim was to improve that design to make it highly available and secure while also making it scalable. In this assignment a custom AMI called LabRole, which was created for us was implemented so that the EC2 instance, S3 bucket, and lambda function can interact securely. An AMI was created and used in a launch template for an auto scaling group which allows new EC2 instances to automatically launch when demand rises. An application load balancer was added to evenly distribute traffic across multiple servers. The lambda function called CreateThumbnail was implemented to automatically resize images after they get uploaded to the S3 bucket. The S3 bucket had access restricted via a custom bucket policy with extra security through the network ACL and security groups. This report outlines how the updated photo album website was designed and deployed using AWS.

II. Infrastructure Deployment

A. Virtual Private Cloud

A custom VPC named JHessVPC was created in us-east-1 with two availability zones, us-east1a and us-east1b, set up the exact same as assignment 1B. Each zone contained a public and private subnet. Each private and public subnet was set up with the correct IPv4 CIDR as specified. The public subnets were associated with a public routing table linked to an Internet Gateway, while both private subnets where in their own route table attached to the NAT gateway.

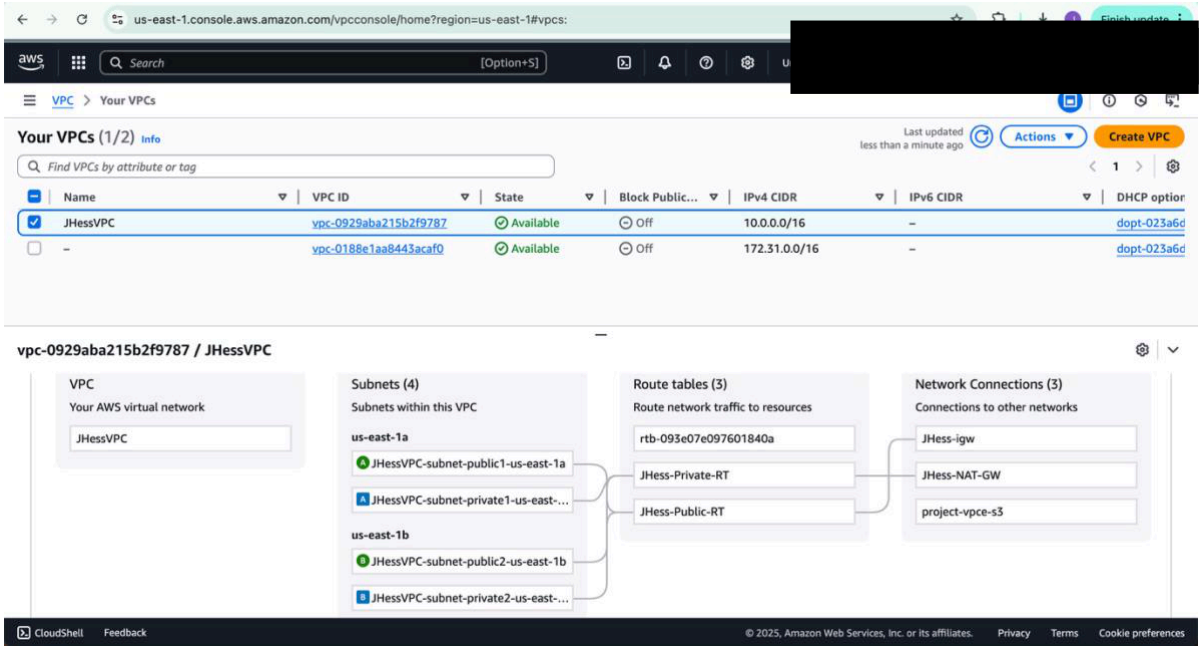


Figure 1: VPC resource map

B. NAT Gateway

A NAT Gateway was created in public subnet 1 to allow instances in the private subnets to securely access the internet which helped with communication with the S3 bucket and lambda. The NAT Gateway was attached to an elastic ip and associated with the private route table so all outbound traffic from private instances was routed through it.

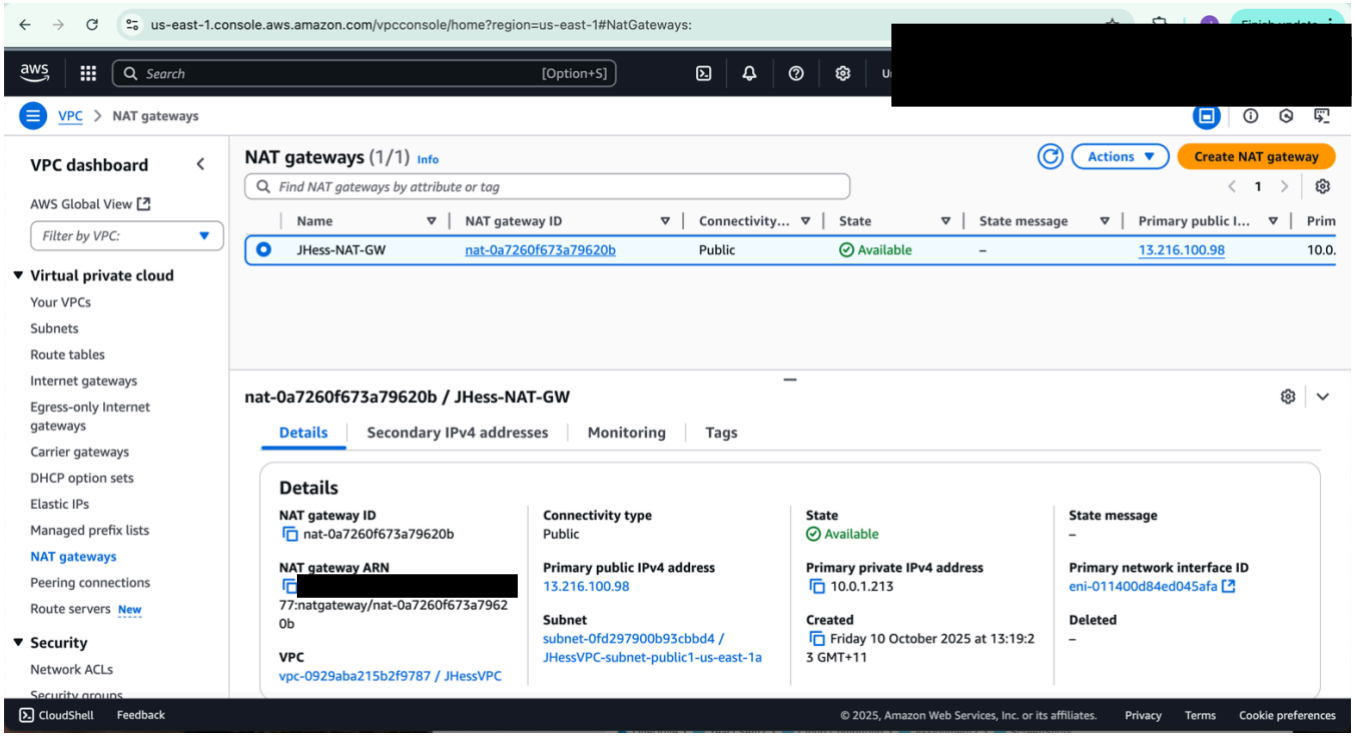


Figure 2: Nat Gateway

C. Security Groups and Network ACL

Four security groups were created to control traffic between all layers of the architecture. These groups include ELBSG, WebServerSG, DBServerSG, DevServerSG. Each security group was configured with least privilege to ensure only necessary communication is allowed. The ELBSG allowed inbound HTTP traffic from the internet, forwarding it to WebServerSG. WebServerSG only accepted traffic from the ELB, while the DBServerSG allowed inbound traffic from MySQL only from the webservers. The NACL named PrivateSubnetsNACL was applied to the private subnets to add another layer of protection. It blocked all ICMP traffic both inbound and outbound between the Dev server and private web servers .

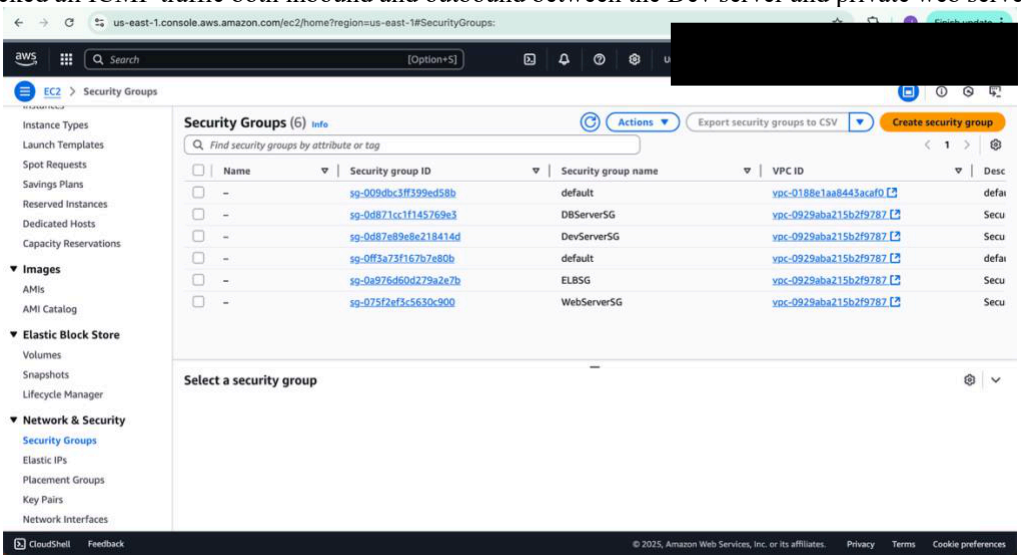


Figure 3: Security Groups

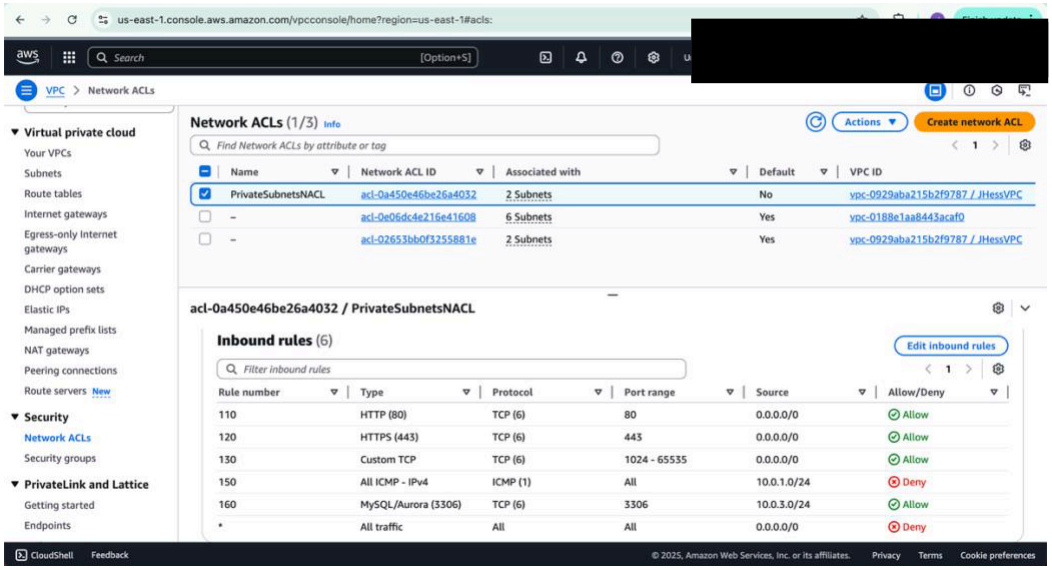


Figure 4: Network ACL

D. *Identity and Access Management (IAM)*

To create secure communication between the AWS services, a custom IAM role created for us called LabRole was used to implement least privilege. This IAM role was attached to the EC2 instances which allowed them to upload photos to the S3 bucket and invoke the Lambda function. It was also applied to the auto scaling group to ensure every new EC2 instance automatically.

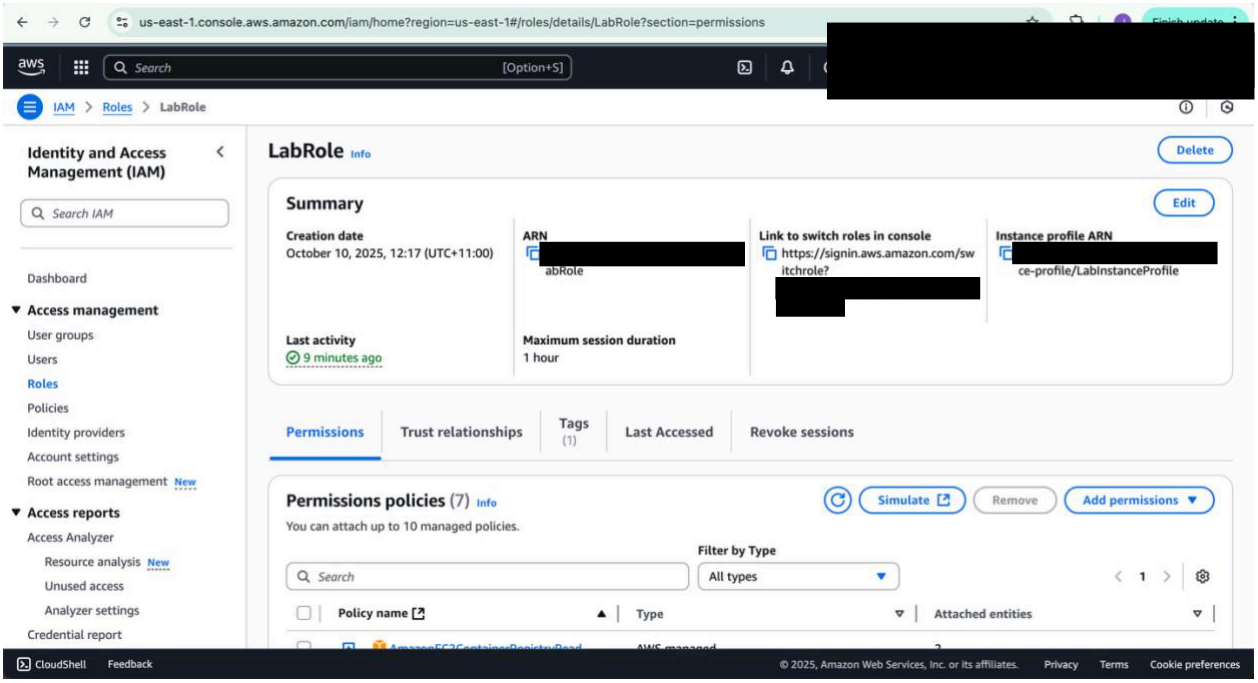


Figure 5: IAM role

III. Application Layer Deployment

A. *EC2 and Custom AMI Deployment*

A development server (Dev Server) was launched to configure and test the photo album application before created a reusable image. This EC2 instance had Apache, PHP, phpMyAdmin, and the AWS SDK for PHP were installed on this instance. Once the dev server was fully configured, a custom Amazon Machine Image was created in order to preserve all the settings and dependencies.

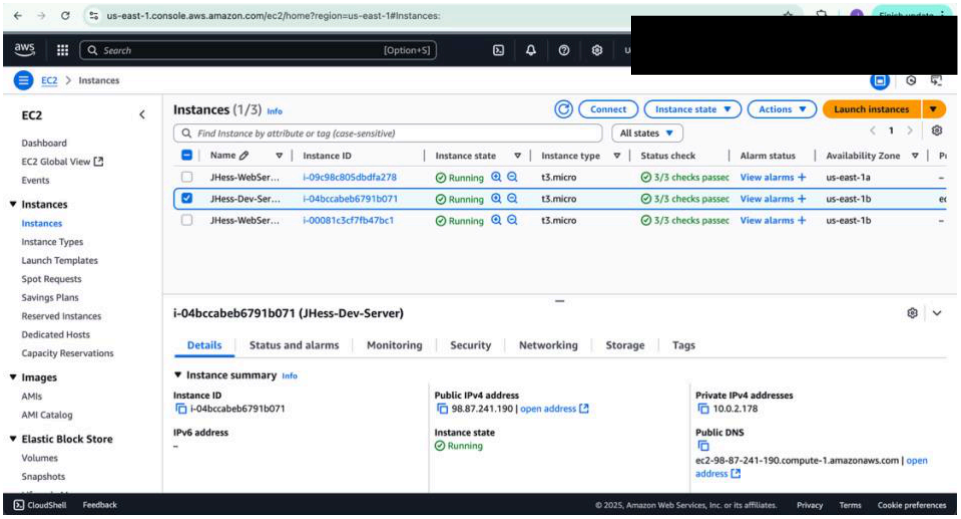


Figure 6: EC2 instance

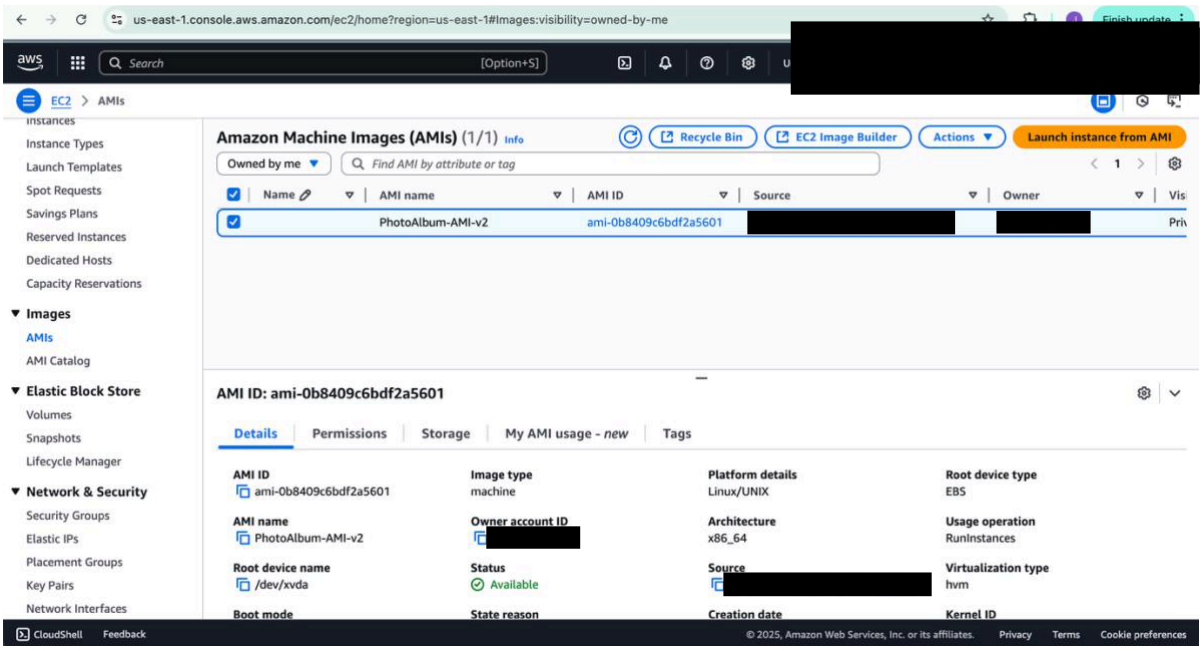


Figure 7: AMI

B. *Launch Template and Auto Scaling Group*

A launch template was created from the AMI created in the previous step. This template was used to deploy an auto scaling group (ASG) which provided fault tolerance and high availability for the architecture. The ASG was configured with a minimum of two instances and a maximum of three with a target tracking policy to maintain an average of 30 requests per target. Whenever demand increases, the ASG automatically launches more EC2 instances, when it decreases it terminates unnecessary instance to reduce credit spending.

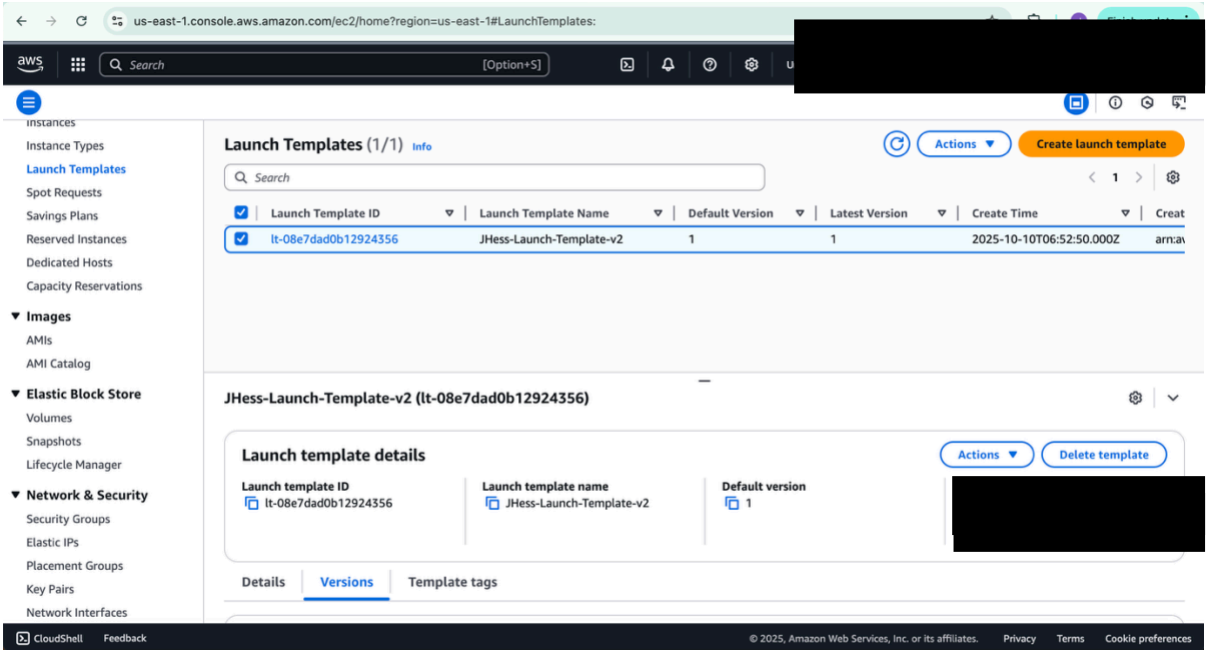


Figure 8: Launch Template

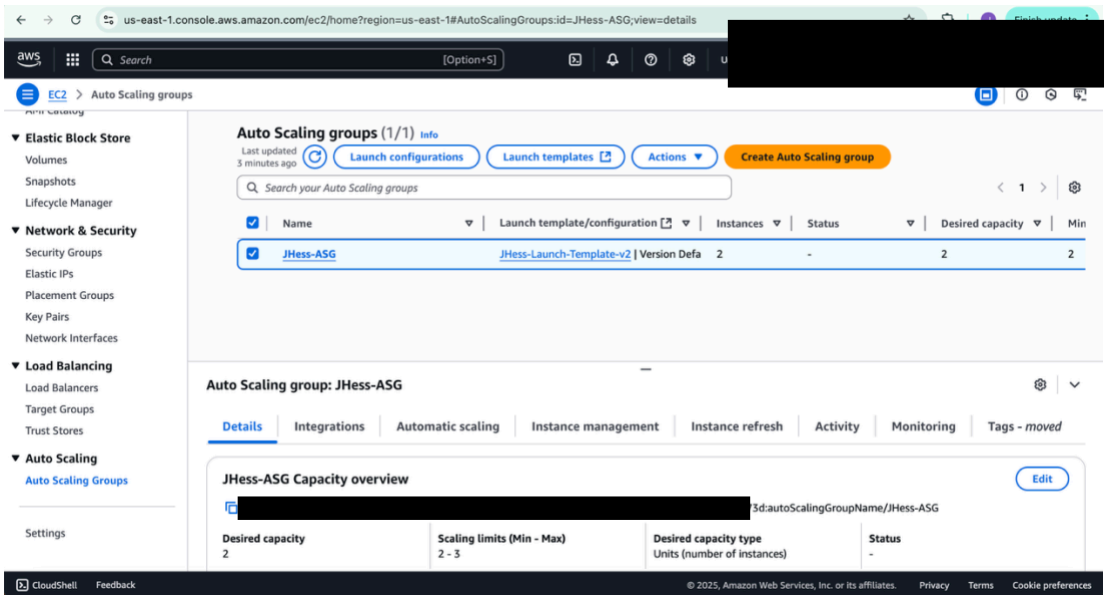


Figure 9: Auto Scaling Group

C. *Elastic Load Balancer*

An application load balancer (ALB) was deployed to distribute web traffic across multiple EC2 instances in the Auto Scaling Group. The ALB was placed in the public subnets and configured with a target group which contatined the private web servers. Health checks constantly monitored each instances availability. Only HTTP traffic from the internet was allowed thorugh the ELBSG which then forwarded requests to the web servers in the private subnets. If an instance fails a health check, the ALB reroutes the request to healthy targets to make sure it's uninterrupted. The load balancer also provided a public DNS through which the photo album website is accessible.

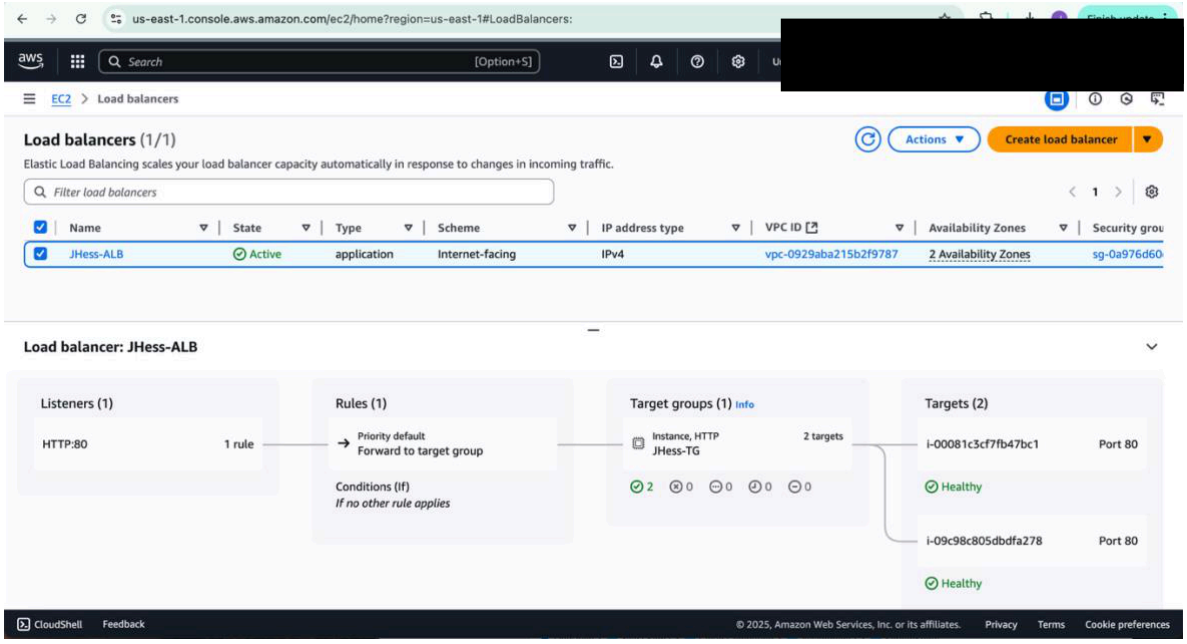


Figure 10: Load Balancer

IV. Storage and Function Integration

A. *Amazon S3 Bucket*

The Amazon S3 bucket created had the same configuration as assignment 1B and was made to support secure and automated photo storage. The bucket stores all uploaded images from the photo album website as well as the resized versions generated by the lambda function.

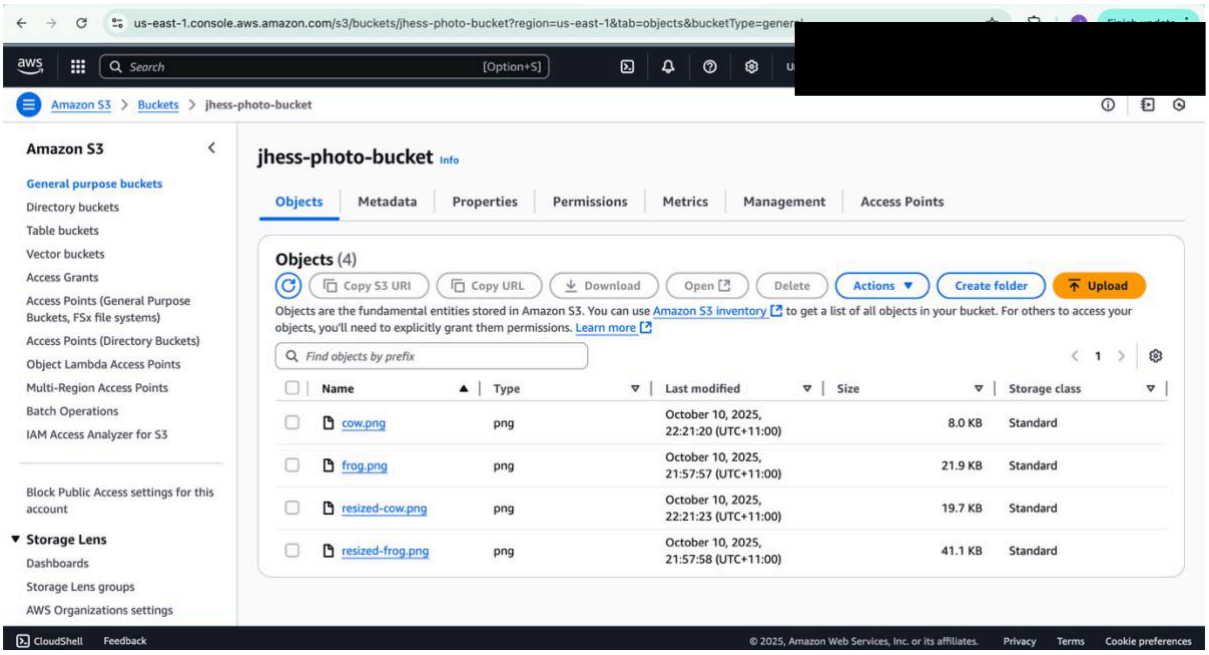


Figure 11: S3 Bucket

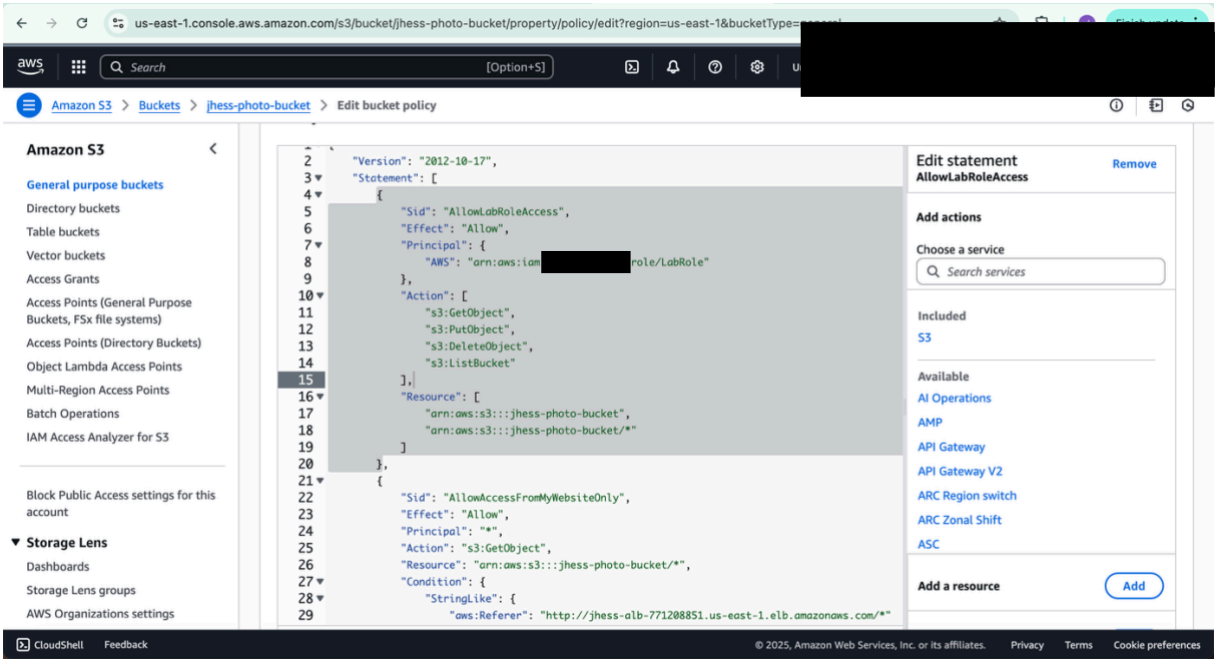


Figure 12: S3 Bucket Policy

B. AWS Lambda Function

A lambda function called CreateThumbnail was used to resize copies of any image uploaded to the S3 bucket. It downloads the image from the S3 bucket, resizes it and then uploads the new image back to the bucket.

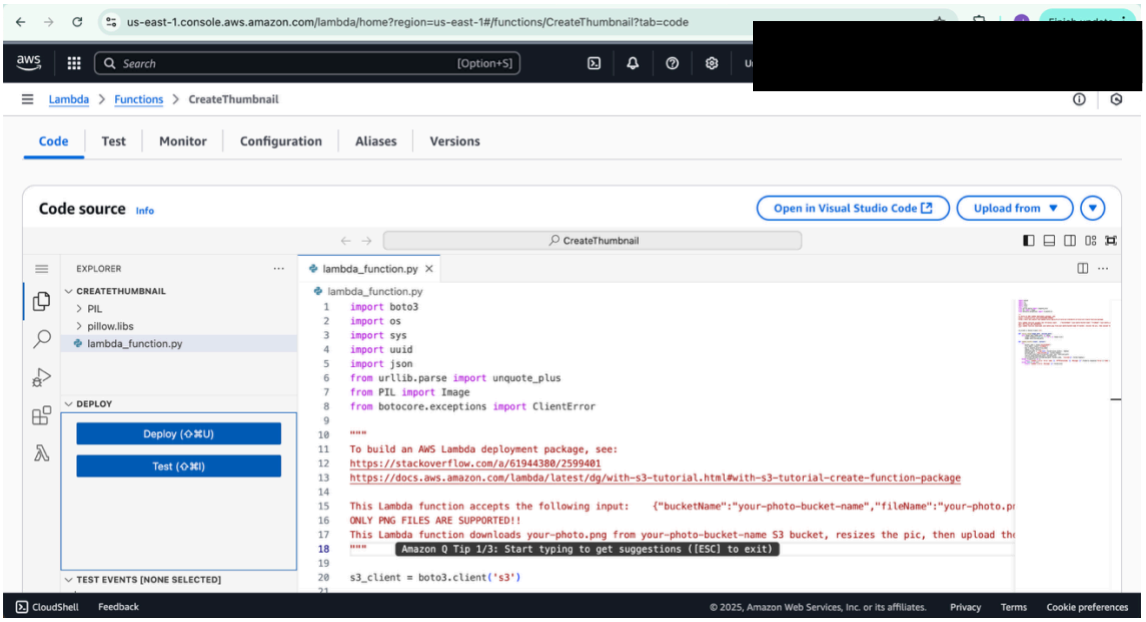


Figure 12: Lambda Function

C. RDS Database

The Amazon RDS MySQL database was recreated from assignment 1B and is used to store photo metadata such as the titles, descriptions, creation date, keywords and object url from the S3 bucket. The RDS instance was deployed within the private subnets for security. A security group called DBServerSG was attached in order to allow inbound MySQL traffic only from the web server which prevents external access. The Dev Server in the public subnet was used to manage the database through phpMyAdmin which meant we didn't have to expose the database directly to the internet.

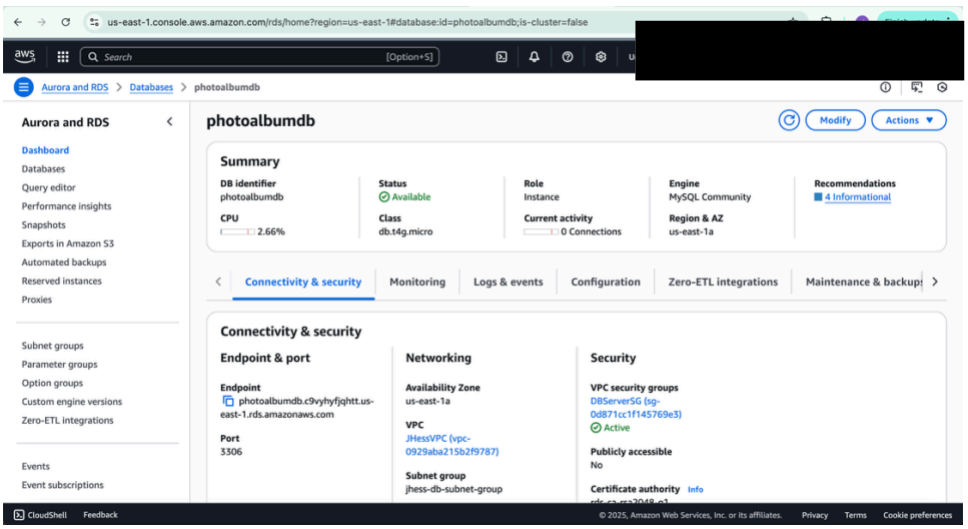


Figure 12: RDS Database

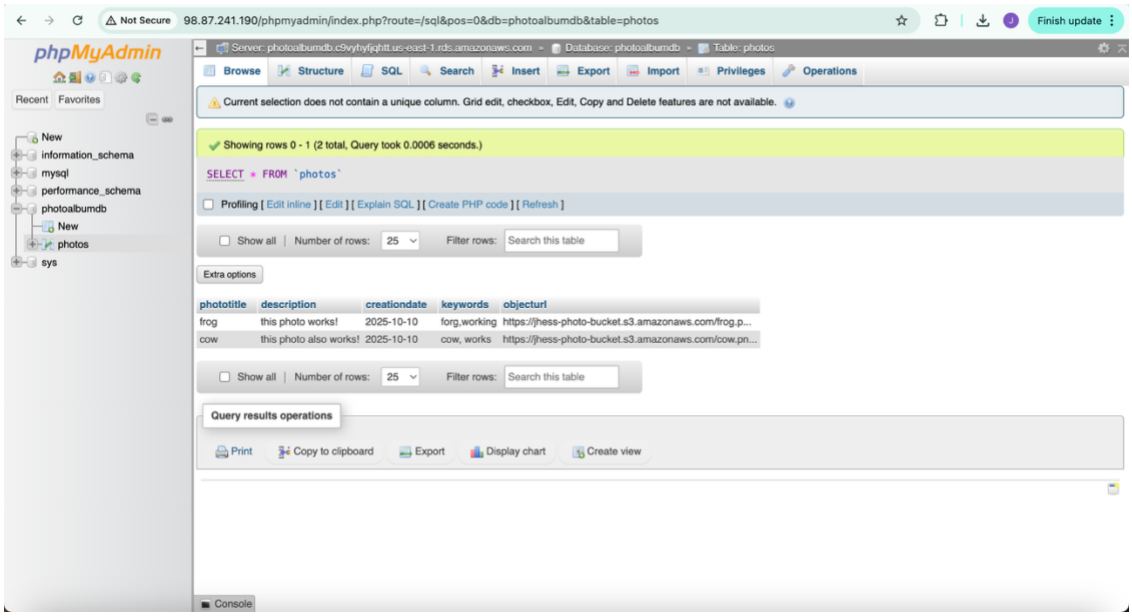


Figure 13: phpMyAdmin database

V. Testing and Validation

The photo album website was tested through the application load balancer in order to confirm that traffic was correctly distributed between multiple EC2 instances in the auto scaling group. All targets showed up as healthy, and new instances launched automatically when one was terminated which proved that the config worked. Photo uploads were tested using the website for photouploader and the photos were successfully stored in the S3 bucket alongside their resized version and also in the RDS database. This also proved the Lambda function worked as intended as the images were resized. Ping tests verified that the Network ACL blocked ICMP traffic between the dev server and private subnets.

VI. Challenges and Achievements

Multiple challenges were encountered in this assignment. One major issue was the S3 bucket policy and ensuring that the correct requirements were included. I achieved this by including a referrer condition that only allowed access through the websites ELB public DNS url. Another challenge occurred when the Lambda function failed to trigger and didn't resize some image. However this was not an issue on my end as it was said in class that some images will not work and some will, I found this to be the case with some images did and some didn't trigger the lambda function.

VII. Credit Management

In order to manage my given credits, all resources were only kept running during deployment, testing, and validation. Instances within the auto scaling group and dev server were stopped straight after each testing session to prevent overuse and over spending. The lambda function and S3 bucket both operate on command which only causes minimal cost.

VIII. Conclusion

This assignment successfully built upon the original photo album website from assignment 1B and turned it into a highly available and scalable architecture. By implementing services such as auto scaling, load balancing, custom IAM roles, and a lambda function, the system can now handle different sized workloads while automatically maintaining security and reliability. All AWS services worked together as intended which demonstrates a well-structured and cost-efficient AWS solution.

IX. References

AWS. (2020). *IAM roles*. Amazon.com. https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html

AWS Lambda deployment package in Python - AWS Lambda. (n.d.). Docs.aws.amazon.com. https://docs.aws.amazon.com/lambda/latest/dg/python-package.html

Bucket policy examples - Amazon Simple Storage Service. (n.d.). Docs.aws.amazon.com. https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies.html#example-

Health checks for your target groups - Elastic Load Balancing. (n.d.). Docs.aws.amazon.com. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html

(2025a). Vocareum.com. https://labs.vocareum.com/main/main.php?m=clabide&mode=s&asnid=4407647&stepid=4407648&hideNavBar=1

(2025b). Vocareum.com. https://labs.vocareum.com/main/main.php?m=clabide&mode=s&asnid=4407645&stepid=4407646&hideNavBar=1